

Z3-Noodler and Mata: String Solving with Stabilization and Transducers

Alpine Verification Meeting 2026

David Chocholatý¹, **Vojtěch Havlena¹**, **Lukáš Holík^{1,2}**, **Juraj Síč¹**, **Michal Šedý²**

¹Faculty of Information Technology, Brno University of Technology, Brno, Czech Republic
{ichocholatý, ihavlena, holik, sicjuraj}@fit.vut.cz

²The Technical Faculty of IT and Design, Aalborg University, Aalborg, Denmark
{lukasholik, medy}@cc.aau.dk

SMT String constraint solving

- Checking **satisfiability** of formulae with **string variables** and **string operations**

$$\underbrace{x = yz \wedge y \neq u}_{(dis)equations} \wedge \underbrace{x \in (ab)^* a^+ (b|c)}_{regular\ constraints} \wedge \underbrace{|x| = 2|u| + 1}_{length\ constraints} \wedge \underbrace{\text{contains}(u, \text{substr}(z, 3, 2)) \wedge \dots}_{more\ complex\ operations}$$

SMT String constraint solving

- Checking **satisfiability** of formulae with **string variables** and **string operations**

$$\underbrace{x = yz \wedge y \neq u \wedge x \in (ab)^* a^+ (b|c)}_{(dis)equations} \wedge \underbrace{(ab)^* a^+ (b|c)}_{regular\ constraints} \wedge \underbrace{|x| = 2|u| + 1}_{length\ constraints} \wedge \underbrace{\text{contains}(u, \text{substr}(z, 3, 2)) \wedge \dots}_{more\ complex\ operations}$$

- **Motivation: large and complex real-world programs** need security guarantees

- **analysis** of string manipulating programs (**vulnerabilities** in web applications)

```
let x = y.substring(1, y.length - 1);    x0 = substr(y, 1, |y| - 1) ∧  
let z = y.concat(x);                    z0 = y · x0 ∧  
assert(x === z);                        x0 ≠ z0
```

- Amazon web services: cloud **access control policies**

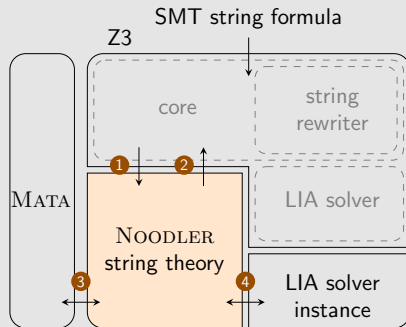
[Rungta-CAV'22]

```
action: deactivate,                      A = "deactivate" ∧  
resource: (a1, a2),                     (R = "a1" ∨ R = "a2") ∧  
condition: {StringLike, s3:prefix, home*}  prefix ∈ home*
```

- **verification** of safety-critical SW: aerospace, automotive, embedded systems, etc.

Z3-Noodler: SMT string solver

- Based on SMT solver **Z3**
 - formula **parsed** by Z3 and handled by **CDCL(*T*)**-based framework
 - Z3-Noodler replaces Z3's **string theory solver**
 - modified **string rewriter** (simplifications)
 - uses **Z3's linear arithmetic** (LIA) theory solver
- Uses **nondeterministic finite automata** (NFAs)
 - library **Mata**¹ for efficient handling of finite automata and operations
 - **Explicit alphabets** sufficient



The fastest string solver: the **winner** of SMT-COMP'24/25/26 string division

¹Chocholatý, D. et al. Mata: A Fast and Simple Finite Automata Library. In: TACAS'24

Timeline



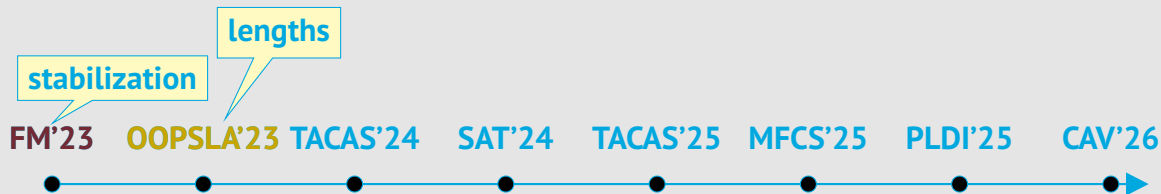
Timeline



FM'23

- Introduces the **stabilization** algorithm
- For **equations** with **regular constr.**
- **Automata**-based
- Just a Python prototype

Timeline



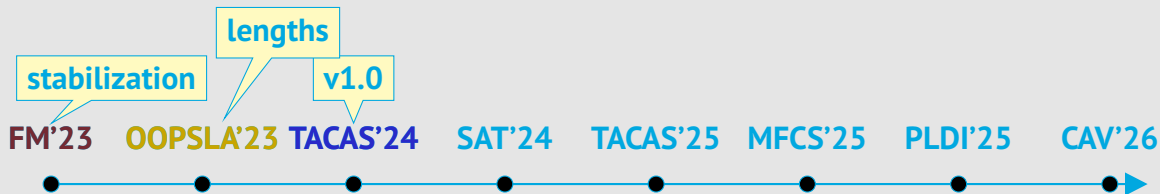
FM'23

- Introduces the **stabilization** algorithm
- For **equations** with **regular constr.**
- **Automata**-based
- Just a Python prototype

OOPSLA'23

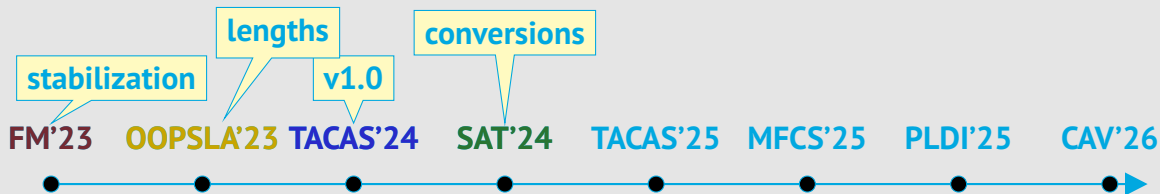
- Combines FM'23 with **splitting**
- Can handle **length constr.**
- Uses **LIA solver** for lengths
- The beginning of **Z3-Noodler**

Timeline



TACAS'24: Tool paper for Z3-Noodler v1.0

Timeline

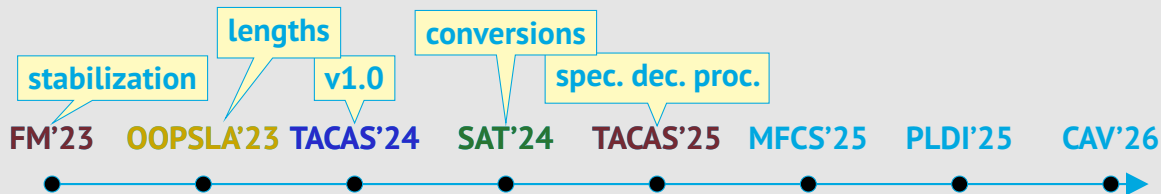


TACAS'24: Tool paper for Z3-Noodler v1.0

SAT'24

- Extends OOPSLA'23 procedure with handling **string-integer conversions**
 - **to_int/from_int** - string to/from integer:
 $\text{to_int}('0324') = 324$ $\text{to_int}('34a') = -1$ $\text{from_int}(134) = '134'$
 - **to_code/from_code** - char to/from (Unicode) code point:
 $\text{to_code}('0') = 48$ $\text{from_code}(97) = 'a'$ $\text{to_code}('ab') = -1$
- Conversions are encoded into **LIA formulae** solved by Z3's LIA solver

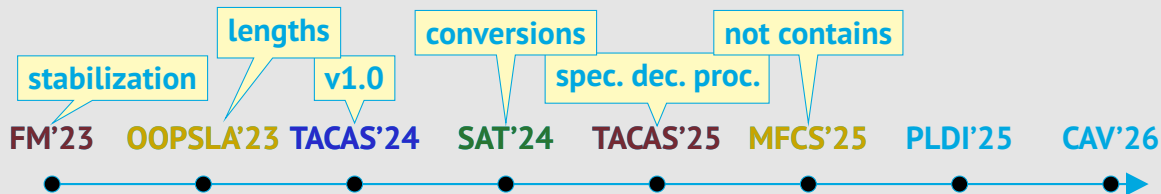
Timeline



TACAS'25

- Complements OOPSLA'23 procedure with **specialized decision procedures**
 - better handling of **pure regular constraints**
 - **Nielsen transformation** for quadratic equations
 - **length-based procedure**
- **Model generation** for all procedures (even FM'23)

Timeline



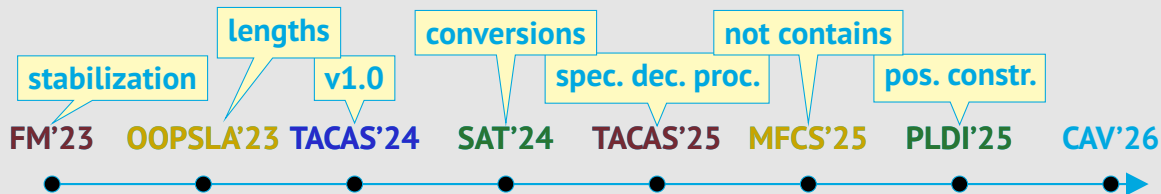
TACAS'25

- Complements OOPSLA'23 procedure with **specialized decision procedures**
 - better handling of **pure regular constraints**
 - **Nielsen transformation** for quadratic equations
 - **length-based procedure**
- **Model generation** for all procedures (even FM'23)

MFCS'25

- Better support of **negated contains**

Timeline



TACAS'25

- Complements OOPSLA'23 procedure with **specialized decision procedures**
 - better handling of **pure regular constraints**
 - **Nielsen transformation** for quadratic equations
 - **length-based procedure**
- **Model generation** for all procedures (even FM'23)

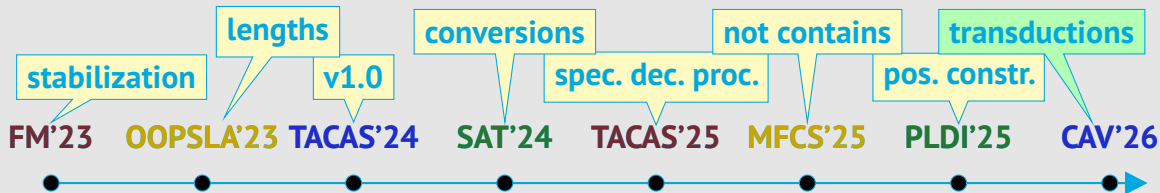
MFCS'25

- Better support of **negated contains**

PLDI'25

- Procedure for **positional constraints**

Timeline



CAV'26 (this work)

- Support for **transductions**

(replacements operations: **replace_all** and **replace_re_all**)

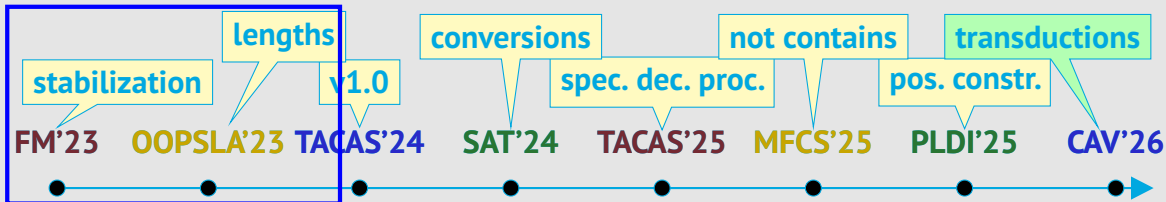
`replace_all('abcdaab', 'ab', 'e') = 'ecdae'` `replace_re_all('abdc', [ab|c], '') = 'd'`

- can be encoded using **finite transducers** (multitape automata; input/output tapes)
- stabilization procedure has to be **extended with transducers**

- **Applications** for Z3-Noodler

- ongoing work with **Honeywell Aerospace**
- analysis of **policies** defined in **Rego** language

Timeline



CAV'26 (this work)

■ Support for **transductions**

(replacements operations: **replace_all** and **replace_re_all**)

`replace_all('abcdaab', 'ab', 'e') = 'ecdae'` `replace_re_all('abdc', [ab|c], '') = 'd'`

- can be encoded using **finite transducers** (multitape automata; input/output tapes)
- stabilization procedure has to be **extended with transducers**

■ **Applications** for Z3-Noodler

- ongoing work with **Honeywell Aerospace**
- analysis of **policies** defined in **Rego** language

Stabilization on an example

$$xyx = zu \wedge ww = xa \wedge u \in (baba)^*a \wedge z \in a(ba)^* \wedge x \in \Sigma^* \wedge y \in \Sigma^* \wedge w \in \Sigma^*$$

■ $\Sigma = \{a, b\}$

Stabilization on an example

$$xyx = zu \wedge ww = xa \wedge u \in (baba)^*a \wedge z \in a(ba)^* \wedge x \in \Sigma^* \wedge y \in \Sigma^* \wedge w \in \Sigma^*$$

- $\Sigma = \{a, b\}$
- Regular constraints are **collected** in a **language assignment** represented by **automata**

$$Lang = \{u \mapsto (baba)^*a, z \mapsto a(ba)^*, x \mapsto \Sigma^*, y \mapsto \Sigma^*, w \mapsto \Sigma^*\}$$

Stabilization on an example

$$xyx = zu \quad ww = xa \quad u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto \Sigma^* \quad y \mapsto \Sigma^* \quad w \mapsto \Sigma^*$$

- $\Sigma = \{a, b\}$
- Regular constraints are **collected** in a **language assignment** represented by **automata**

$$Lang = \{u \mapsto (baba)^*a, z \mapsto a(ba)^*, x \mapsto \Sigma^*, y \mapsto \Sigma^*, w \mapsto \Sigma^*\}$$

Stabilization on an example

$$xyx = zu \quad ww = xa \quad u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto \Sigma^* \quad y \mapsto \Sigma^* \quad w \mapsto \Sigma^*$$

- $\Sigma = \{a, b\}$
- Regular constraints are **collected** in a **language assignment** represented by **automata**

$$Lang = \{u \mapsto (baba)^*a, z \mapsto a(ba)^*, x \mapsto \Sigma^*, y \mapsto \Sigma^*, w \mapsto \Sigma^*\}$$

- Use equations to **refine** *Lang*, starting with **$xyx = zu$**
- For any solution (assignment ν) string $s = \nu(x) \cdot \nu(y) \cdot \nu(x) = \nu(z) \cdot \nu(u)$ satisfies:

$$s \in \underbrace{\Sigma^*}_x \underbrace{\Sigma^*}_y \underbrace{\Sigma^*}_x = \underbrace{a(ba)^*}_z \underbrace{(baba)^*a}_u$$

- Use right side to **refine** languages of variables x, y on the left side by **noodlification**

Stabilization on an example

$xyx = zu$ $ww = xa$	$u \mapsto (baba)^*a$ $z \mapsto a(ba)^*$ $x \mapsto \Sigma^*$ $y \mapsto \Sigma^*$ $w \mapsto \Sigma^*$
----------------------	--

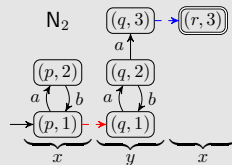
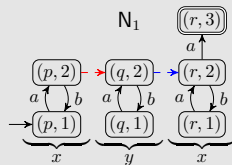
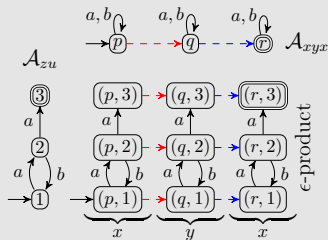
- Use right side to **refine** languages of variables x, y on the left side by **noodlification**

Stabilization on an example

$$xyx = zu \quad ww = xa \quad u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto \Sigma^* \quad y \mapsto \Sigma^* \quad w \mapsto \Sigma^*$$

- Use right side to **refine** languages of variables x, y on the left side by **noodlification**
- Leads to two **noodles**:

$$N_1 : \underbrace{\Sigma^*}_x \underbrace{\Sigma^*}_y \underbrace{\Sigma^*}_x \cap \underbrace{a(ba)^*}_z \underbrace{(baba)^*a}_u \quad N_2 : \underbrace{\Sigma^*}_x \underbrace{\Sigma^*}_y \underbrace{\Sigma^*}_x \cap \underbrace{a(ba)^*}_z \underbrace{(baba)^*a}_u$$



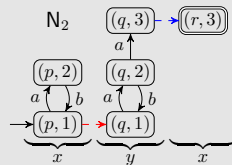
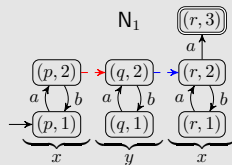
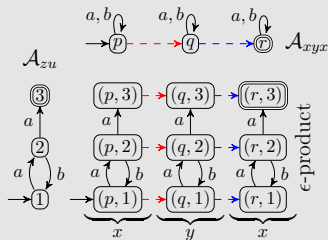
Stabilization on an example

$$xyx = zu \quad ww = xa \quad u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto \Sigma^* \quad y \mapsto \Sigma^* \quad w \mapsto \Sigma^*$$

- Use right side to **refine** languages of variables x, y on the left side by **noodlification**
- Leads to two **noodles**:

$$N_1 : \overbrace{a}^x \overbrace{(ba)^*}^y \overbrace{a}^x = \overbrace{a(ba)^*}^z \overbrace{(baba)^*a}^u$$

$$N_2 : \overbrace{\Sigma^*}^x \overbrace{\Sigma^*}^y \overbrace{\Sigma^*}^x = \overbrace{a(ba)^*}^z \overbrace{(baba)^*a}^u$$



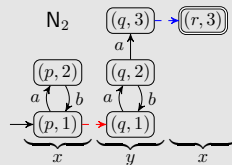
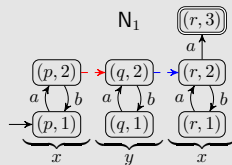
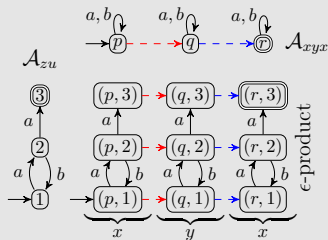
Stabilization on an example

$$xyx = zu \quad ww = xa \quad u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto \Sigma^* \quad y \mapsto \Sigma^* \quad w \mapsto \Sigma^*$$

- Use right side to **refine** languages of variables x, y on the left side by **noodlification**
- Leads to two **noodles**:

$$N_1 : \overbrace{a}^x \overbrace{(ba)^*}^y \overbrace{a}^x = \overbrace{a(ba)^*}^z \overbrace{(baba)^*a}^u$$

$$N_2 : \overbrace{\epsilon}^x \overbrace{a(ba)^*a}^y \overbrace{\epsilon}^x = \overbrace{a(ba)^*}^z \overbrace{(baba)^*a}^u$$



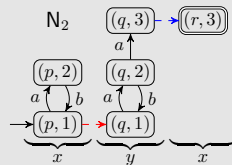
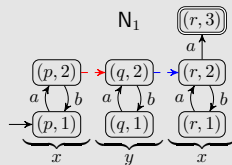
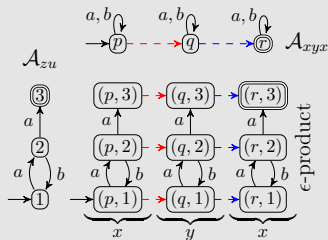
Stabilization on an example

$$xyx = zu \quad ww = xa \quad \begin{array}{l} u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto a \quad y \mapsto (ba)^* \quad w \mapsto \Sigma^* \end{array}$$

- Use right side to **refine** languages of variables x, y on the left side by **noodlification**
- Leads to two **noodles**:

$$N_1 : \overbrace{a}^x \overbrace{(ba)^*}^y \overbrace{a}^x = \overbrace{a(ba)^*}^z \overbrace{(baba)^*a}^u$$

$$N_2 : \overbrace{\epsilon}^x \overbrace{a(ba)^*a}^y \overbrace{\epsilon}^x = \overbrace{a(ba)^*}^z \overbrace{(baba)^*a}^u$$



- We choose the first one N_1 and update the languages of x and y

Stabilization on an example

$$xyx = zu \quad \mathbf{ww = xa} \quad \left| \quad u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto a \quad y \mapsto (ba)^* \quad w \mapsto \Sigma^*$$

- Refine further with $\mathbf{ww = xa}$:

$$\overbrace{\Sigma^*}^w \cap \overbrace{\Sigma^*}^w = \overbrace{a}^x \overbrace{a}^a.$$

Stabilization on an example

$xyx = zu$ $ww = xa$	$u \mapsto (baba)^*a$ $z \mapsto a(ba)^*$ $x \mapsto a$ $y \mapsto (ba)^*$ $w \mapsto a$
----------------------	--

- Refine further with $ww = xa$:

$$\underbrace{\quad}_w \underbrace{\quad}_w = \underbrace{\quad}_x \quad a$$

$a \quad a \quad \cap \quad a \quad a.$

Stabilization on an example

$xyx = zu \quad ww = xa$	$u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto a \quad y \mapsto (ba)^* \quad w \mapsto a$
--------------------------	--

- Refine further with **$ww = xa$** :

$$\underbrace{\quad}_a^w \underbrace{\quad}_a^w = \underbrace{\quad}_a^x a.$$

- Languages in equations now **match**:

$$\underbrace{\quad}_a^x \underbrace{\quad}_{(ba)^*}^y \underbrace{\quad}_a^x = \underbrace{\quad}_{a(ba)^*}^z \underbrace{\quad}_{(baba)^*a}^u \quad \text{and} \quad \underbrace{\quad}_a^w \underbrace{\quad}_a^w = \underbrace{\quad}_a^x a.$$

Stabilization on an example

$xyx = zu \quad ww = xa$	$u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto a \quad y \mapsto (ba)^* \quad w \mapsto a$
--------------------------	--

- Refine further with $ww = xa$:

$$\underbrace{\quad}_a^w \underbrace{\quad}_a^w = \underbrace{\quad}_a^x a.$$

- Languages in equations now **match**:

$$\underbrace{\quad}_a^x \underbrace{\quad}_{(ba)^*}^y \underbrace{\quad}_a^x = \underbrace{\quad}_{a(ba)^*}^z \underbrace{\quad}_{(baba)^*a}^u \quad \text{and} \quad \underbrace{\quad}_a^w \underbrace{\quad}_a^w = \underbrace{\quad}_a^x a.$$

- $Lang$ is a **stable solution** (we prove this is enough to decide it is SAT)

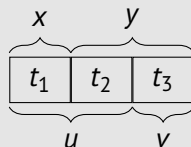
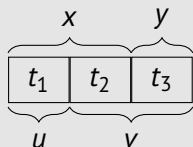
Handling complex string operations, disequations, and lengths

$$\underbrace{x = yz \wedge y \neq u}_{\text{(dis)equations}} \wedge \underbrace{x \in (ab)^* a^+ (b|c)}_{\text{regular constraints}} \wedge \underbrace{|x| = 2|u| + 1}_{\text{length constraints}} \wedge \underbrace{\text{contains}(u, \text{substr}(z, 3, 2)) \wedge \dots}_{\text{(some) more complex operations}}$$

- FM'23 can handle **equations** and **regular constraints**
- How to handle more **complex operations** and **disequations**?
 - ↪ reduced (at least partially) to simpler constraints
- How to handle **lengths**?
 - create linear-integer arithmetic (LIA) formula **encoding possible lengths of words** for each language in *Lang*
 - stable solution *Lang* does not keep **dependencies** between lengths of vars
 - ↪ we use noodlification combined with **Align&Split** algorithm [Abdulla-CAV'14]

Handling lengths: Align&Split on an example

- The equation $xy = uv$ gives us two alignments and splits:



- t_1 , t_2 , and t_3 are **fresh variables**
- We can use **modified noodlification** to get their languages
- Original variables **substituted** by fresh variables, giving us **exact** characterization:

$$x \mapsto t_1 t_2$$

$$y \mapsto t_3$$

$$u \mapsto t_1$$

$$v \mapsto t_2 t_3$$

$$x \mapsto t_1$$

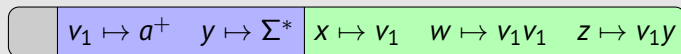
$$y \mapsto t_2 t_3$$

$$u \mapsto t_1 t_2$$

$$v \mapsto t_3$$

Handling lengths: Computing models

■ $xx = w \wedge z = xy \wedge w \in a^+ \wedge |z| = 2|w| - |x|$



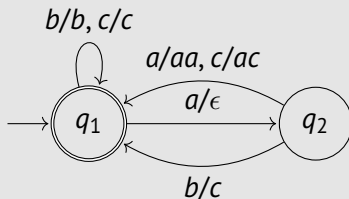
- **LIA formula** encoding **possible lengths** of variables:

$$\varphi_{\text{len}} \stackrel{\text{def.}}{\Leftrightarrow} |v_1| \geq 1 \wedge |y| \geq 0 \wedge |x| = |v_1| \wedge |w| = |v_1| + |v_1| \wedge |z| = |v_1| + |y|$$

- Ask LIA solver if $|z| = 2|w| - |x| \wedge \varphi_{\text{len}}$ is satisfiable
- it is, we have model $|v_1| = |x| = 1, |w| = |y| = 2, |z| = 3$
 - we can choose any word from $\text{Lang}(v_1)$ and $\text{Lang}(y)$ with correct lengths:
 $v_1 = a$ and $y = bc$
 - models for x, w , and z are computed using the substitution map σ :
 $x = v_1 = a, w = v_1 v_1 = aa$, and $z = v_1 y = abc$

Adding transductions (replacement operations, ...)

- How to add **replace_all** and **replace_re_all** to stabilization?
- **Emplacement** only as a **string literal**; **lazy left-most matching** (SMT-LIB semantics):
 $xyx = \text{replace_all}(uv, ('ab')^+, 'c')$
- Can be represented as a **transduction (transducer term)**
 $xyx = T(uv)$ where T is a **transducer**:



- We can **always** transform such a replacement operation to a transducer [Fu-NFM'10]

Stabilization with transductions

$$xyx = T(zu) \quad ww = xa \quad \left| \quad u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto \Sigma^* \quad y \mapsto \Sigma^* \quad w \mapsto \Sigma^* \right.$$

- T represents `replace_all` (`_`, `'ba'`, `'c'`)

Stabilization with transductions

$$xyx = T(zu) \quad ww = xa \quad \begin{array}{l} u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto \Sigma^* \quad y \mapsto \Sigma^* \quad w \mapsto \Sigma^* \end{array}$$

- T represents `replace_all` (`_`, `'ba'`, `'c'`)
- Now both **equations** and **transducers** are used for refinements
- We can start refining with $xyx = T(zu)$:

$$\underbrace{x}_{\Sigma^*} \underbrace{y}_{\Sigma^*} \underbrace{x}_{\Sigma^*} = T\left(\underbrace{z}_{a(ba)^*} \underbrace{u}_{(baba)^*a}\right)$$

Stabilization with transductions

$$xyx = T(zu) \quad ww = xa \quad \begin{array}{l} u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto a \quad y \mapsto c^* \quad w \mapsto \Sigma^* \end{array}$$

- T represents `replace_all` (`_`, `'ba'`, `'c'`)
- Now both **equations** and **transducers** are used for refinements
- We can start refining with $xyx = T(zu)$:

$$\underbrace{a}^x \underbrace{c^*}^y \underbrace{a}^x = \cap T\left(\underbrace{a(ba)^*}^z \underbrace{(baba)^*a}^u\right)$$

Stabilization with transductions

$$xyx = T(zu) \quad ww = xa \quad \begin{array}{l} u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto a \quad y \mapsto c^* \quad w \mapsto \Sigma^* \end{array}$$

- T represents `replace_all` (`_`, `'ba'`, `'c'`)
- Now both **equations** and **transducers** are used for refinements
- We can start refining with $xyx = T(zu)$:

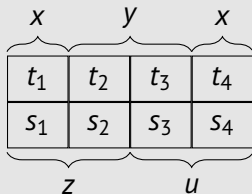
$$\underbrace{x}_{a} \underbrace{y}_{c^*} \underbrace{x}_{a} = \cap T\left(\underbrace{z}_{a(ba)^*} \underbrace{u}_{(baba)^*a}\right)$$

- **Noodlification** needs to be **modified** to handle transducers
 $\rightsquigarrow$ instead of operations on automata, more complex operations on transducers

Lengths with transductions

$$xyx = T(zu) \quad ww = xa \quad \begin{array}{l} u \mapsto (baba)^*a \quad z \mapsto a(ba)^* \quad x \mapsto a \quad y \mapsto c^* \quad w \mapsto \Sigma^* \end{array}$$

- To handle **lengths**, we also apply **modified splitting**:



- s_i/t_i are **fresh input/output variables** and T is split into **simple transducers**:

$$t_1 = T_1(s_1) \quad t_2 = T_2(s_2) \quad t_3 = T_3(s_3) \quad t_4 = T_4(s_4)$$

- **Parikh image** of (combination) of simple transducers **encodes lengths**

Rational Relations

- We also add support for **full rational relations** (our own extension to SMT-LIB standard)
- Transform every "a" to "b" written **elegantly**:

```
(declare-const a_to_b RatRel)
(assert (= a_to_b (rat.* (rat.union (str.to_rat "a" "b")
  (rat.identity (re.diff re.allchar (str.to_re "a"))))))))
```

- Examples:

```
(declare-const s1 String)
(assert (str.in_rat "asbabaa" s1 a_to_b)) ; s1 = "bsbbbbbb"
(declare-const s2 String)
(assert (str.in_rat "dsfgeb" s2 a_to_b)) ; s1 = "dsfgeb"
```

Experimental Evaluation: Replacement benchmarks

Number of solved instances and the time (in seconds) needed to solve them (w/o TOs). Total number of formulae for each benchmark set is given in parentheses.

We use ~ 0 to denote a value close to zero.

	PCP (1,000)		RNA (1,000)		Transducer+ (91)		Webapp (340)		All (2,431)	
	solved	time	solved	time	solved	time	solved	time	solved	time
Z3-Noodler	500	~ 0	1000	8	41	455	338	20	1879	482
cvc5	0	N/A	0	N/A	42	372	293	456	335	827
OSTRICH	479	11,276	1000	8,530	39	613	247	1,913	1765	22,332
Z3	0	N/A	0	N/A	1	~ 0	213	5	214	5

Conclusion and Acknowledgments

■ List of collaborators:

- František Blahoudek
- Yu-Fang Chen
- David Chocholatý
- Vojtěch Havlena
- Michal Hečko
- Lukáš Holík
- Jan Hranička
- Ondřej Lengál
- Juraj Síč



<https://github.com/VeriFIT/z3-noodler>

■ Results:

- Z3-Noodler currently **supports all operations defined by SMT-LIB standard**
- We are working on **applications**